وزارة الاتصالات وتكنولوجيا المعلومات

رواد مصر الرقمية

System Analysis & Design Document: AI-Based Security Monitoring and Anomaly Detection System (Mini SIEM)

Table of Contents

1. Problem Statement & Objectives ……

2. Use Case Diagram & Descriptions …….

3. Functional & Non-Functional Requirements ……

4. Software Architecture …….

5. Database Design & Data Modeling ……

6. Data Flow & System Behavior ……...

7. UI/UX Design & Prototyping …….

8. System Deployment & Integration ……

9. Additional Deliverables ……..

10. References …….

1. Problem Statement & Objectives

1.1. Problem Statement

Traditional security systems, such as conventional Intrusion Detection Systems (IDS), face significant limitations in addressing the evolving landscape of cyber threats. These limitations include an inherent inability to detect novel or zero-day attacks due to their reliance on predefined rules and signatures. Furthermore, these systems struggle with identifying sophisticated insider threats that leverage valid credentials, as such activities often mimic legitimate user behavior. A pervasive issue is the high false positive rate, which leads to alert fatigue among security analysts, diminishing their effectiveness and potentially causing critical alerts to be overlooked [1]. The current paradigm necessitates a reactive approach, often failing to adapt to new attack vectors without constant manual updates.

1.2. Project Objectives

The primary objective of this project is to design and implement a Mini Security Information and Event Management (SIEM) system that leverages Artificial Intelligence (AI) to enhance threat detection capabilities. Specifically, the system aims to:

- Automate Anomaly Detection: Develop a system capable of automatically identifying suspicious and anomalous activities within network traffic and system logs.
- Improve Zero-Day Attack Detection: Utilize unsupervised machine learning models to learn normal system behavior and flag deviations, thereby enabling the detection of unknown or zero-day attacks without prior knowledge of attack patterns.
- Reduce False Positives: Implement anomaly-based detection to significantly lower the rate of false positives compared to traditional rule-based systems, thereby mitigating alert fatigue for security analysts.
- Provide Comparative Analysis: Employ multiple unsupervised anomaly detection models (e.g., Isolation Forest, One-Class Support Vector Machine, Autoencoder Neural Network) to provide robust detection and allow for comparative analysis of their effectiveness.
- Facilitate Security Monitoring: Offer an interactive dashboard for visualizing detected anomalies and alerts, assisting security analysts in real-time monitoring and informed decision-making.
- Establish a Foundation for Real-World Deployment: Create a functional prototype that demonstrates the viability of AI-driven security monitoring, with considerations for future scalability and integration into real-world enterprise environments.

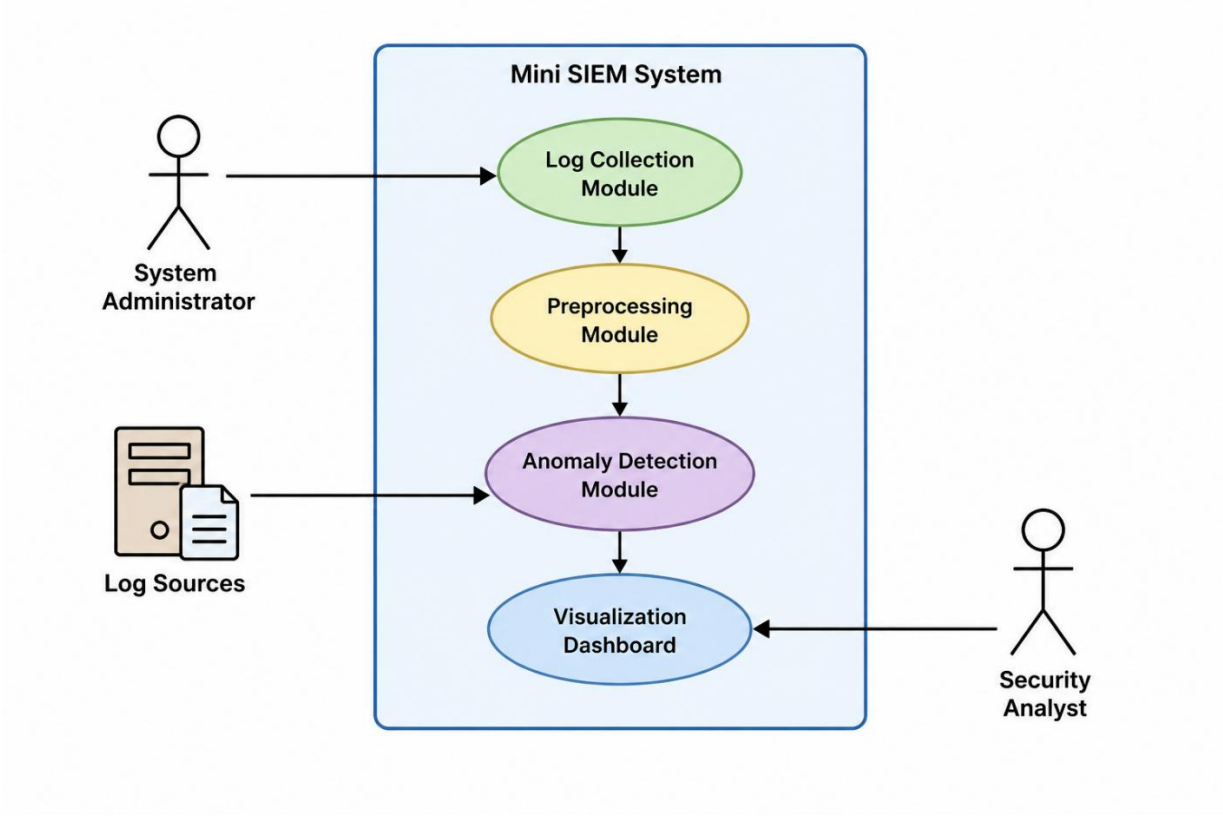
2. Use Case Diagram & Descriptions

2.1. Actors

The primary actors interacting with the Mini SIEM system are:

- Security Analyst: The main user of the system, responsible for monitoring alerts, investigating anomalies, and making security decisions.
- System Administrator: Responsible for deploying, configuring, and maintaining the Mini SIEM system, including log sources and model updates.

2.2. Use Case Diagram



2.3. Use Case Descriptions

Use Case: Monitor Alerts and Anomalies

- Actor: Security Analyst
- Description: The Security Analyst interacts with the Visualization Dashboard to view real-time alerts and detected anomalies. They can filter, sort, and drill down into specific events to understand the nature of the threat.
- Preconditions: The Mini SIEM system is operational, and anomalies have been detected and displayed on the dashboard.
- Postconditions: The Security Analyst has reviewed the alerts and initiated appropriate response actions (outside the scope of this system).

Use Case: Configure System Parameters

- Actor: System Administrator
- Description: The System Administrator configures various parameters of the Mini SIEM system, including log source integrations, machine learning model settings (e.g., thresholds), and dashboard display options.
- Preconditions: The System Administrator has authenticated access to the system configuration interface.
- Postconditions: System parameters are updated and applied, affecting log collection, anomaly detection, and visualization.

Use Case: Maintain System

- Actor: System Administrator
- Description: The System Administrator performs maintenance tasks such as updating machine learning models, managing system resources, and troubleshooting issues to ensure continuous operation and optimal performance of the Mini SIEM system.
- Preconditions: The System Administrator has authenticated access to the system maintenance interface.
- Postconditions: The Mini SIEM system continues to operate efficiently with updated components and resolved issues.

Use Case: Ingest Logs

- Actor: Log Sources (External System)
- Description: Various network devices and system applications generate and send security-relevant logs (e.g., firewall logs, server logs, authentication logs) to the Mini SIEM system for processing.
- Preconditions: Log sources are configured to forward logs to the Mini SIEM's Log Collection Module.
- Postconditions: Logs are successfully collected and queued for preprocessing.

Use Case: Detect Anomalies

- Actor: Mini SIEM System (Internal Process)
- Description: The system continuously processes incoming log data, applies various unsupervised machine learning models (Isolation Forest, OC-SVM, Autoencoder) to identify deviations from learned normal behavior, and flags these deviations as potential anomalies.
- Preconditions: Processed log data is available from the Preprocessing Module.
- Postconditions: Anomalies are identified and forwarded to the Visualization Dashboard for display.

3. Functional & Non-Functional Requirements

3.1. Functional Requirements

The Mini SIEM system shall provide the following functional capabilities:

- FR1: Log Collection: The system shall collect network traffic and system event logs from various configured sources in a structured format.
- FR2: Data Preprocessing: The system shall perform data cleaning, encoding of categorical features, and feature scaling on collected log data.
- FR3: Anomaly Detection: The system shall apply unsupervised machine learning models (Isolation Forest, One-Class Support Vector Machine, Autoencoder Neural Network) to identify anomalous behavior in preprocessed log data.
- FR4: Alert Generation: The system shall generate alerts for detected anomalies, including details such as anomaly score, timestamp, and source information.
- FR5: Interactive Visualization: The system shall provide an interactive dashboard to display real-time alerts and detected anomalies.
- FR6: Anomaly Details Display: The dashboard shall allow security analysts to view detailed information about each detected anomaly.

- FR7: Model Management: The system shall allow System Administrators to configure and update the parameters of the anomaly detection models.
- FR8: Log Source Configuration: The system shall allow System Administrators to configure and manage log sources.

3.2. Non-Functional Requirements

The Mini SIEM system shall adhere to the following non-functional requirements:

- NFR1: Performance: The system shall process incoming log data and detect anomalies with minimal latency to support near real-time monitoring.
- NFR2: Scalability: The system shall be capable of handling an increasing volume of log data and a growing number of log sources without significant degradation in performance.
- NFR3: Reliability: The system shall operate continuously and reliably, minimizing downtime and ensuring consistent anomaly detection.
- NFR4: Security: The system shall ensure the confidentiality, integrity, and availability of collected log data and system configurations. Access to the system and its configurations shall be authenticated and authorized.
- NFR5: Usability: The interactive dashboard shall be intuitive and user-friendly for security analysts to effectively monitor and investigate alerts.
- NFR6: Maintainability: The system's codebase and architecture shall be well-documented and modular to facilitate future updates, bug fixes, and enhancements.
- NFR7: Extensibility: The system shall be designed to allow for the integration of new anomaly detection models or additional log sources in the future.
- NFR8: Data Retention: The system shall support configurable data retention policies for collected logs and anomaly data.

4. Software Architecture

4.1. High-Level Design

The Mini SIEM system will adopt a modular, layered architecture to ensure scalability, maintainability, and extensibility. The architecture separates concerns into distinct components, facilitating independent development and deployment. The core components include a Log Collection Module, a Preprocessing Module, an Anomaly Detection Module, and a Visualization Module.

4.2. Architectural Style

The system will primarily follow a Microservices-oriented architecture for its backend components, allowing for independent scaling and technology choices for each service. The frontend will be a Single Page Application (SPA) interacting with the backend services via RESTful APIs. This approach aligns with the proposed solution's flexibility and potential for real-world deployment, as mentioned in the project idea [1].

4.3. System Components and Interactions

4.3.1. Log Collection Module

- Description: Responsible for ingesting raw log data from various sources (e.g., network devices, servers). It will support different log formats and protocols.
- Key Technologies: Potentially Logstash or Kafka for real-time ingestion, as suggested for real-world deployment [1].
- Interactions: Forwards raw log data to the Preprocessing Module.

4.3.2. Preprocessing Module

- Description: Cleans, normalizes, and transforms raw log data into a format suitable for anomaly detection. This includes tasks like data cleaning, encoding categorical features, and feature scaling.
- Key Technologies: Python with Pandas for data manipulation.
- Interactions: Receives raw logs from the Log Collection Module and sends processed data to the Anomaly Detection Module.

4.3.3. Anomaly Detection Module

- Description: Houses the core AI logic, applying unsupervised machine learning models (Isolation Forest, One-Class SVM, Autoencoder Neural Network) to identify anomalies. It calculates anomaly scores and flags suspicious activities.
- Key Technologies: Python with Scikit-learn for traditional ML models and PyTorch for the Autoencoder Neural Network.
- Interactions: Receives processed data from the Preprocessing Module and sends detected anomalies and alerts to the Visualization Module.

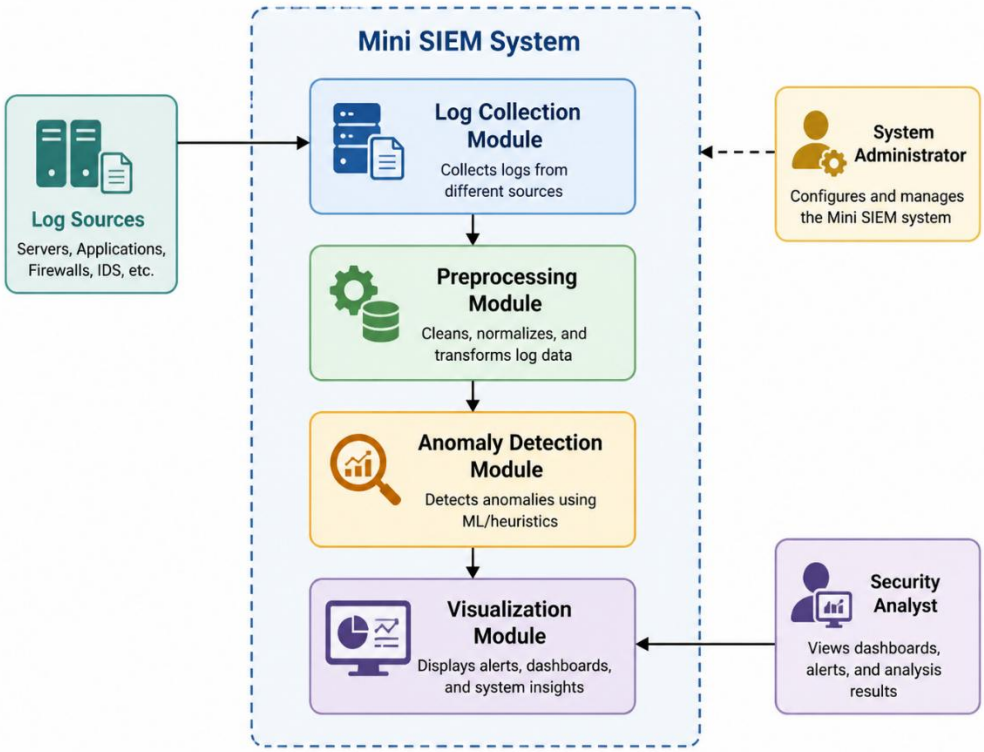
4.3.4. Visualization Module

- Description: Provides an interactive dashboard for security analysts to visualize alerts, anomalies, and system metrics in real-time. It allows for filtering, searching, and drilling down into specific events.
- Key Technologies: Streamlit for rapid prototyping of the dashboard, as chosen for its Python-centric approach [1]. For a real-world scenario, a time-series database like InfluxDB would store the data, and the dashboard would connect to it [1].
- Interactions: Receives anomaly data from the Anomaly Detection Module and interacts directly with the Security Analyst.

4.3.5. API Gateway (Future Consideration)

- Description: Acts as a single entry point for all client requests, routing them to the appropriate microservice. Handles cross-cutting concerns like authentication, rate limiting, and load balancing.
- Key Technologies: FastAPI for wrapping Python models into a REST API [1].
- Interactions: Intercepts requests from the Visualization Module and forwards them to the relevant backend services.

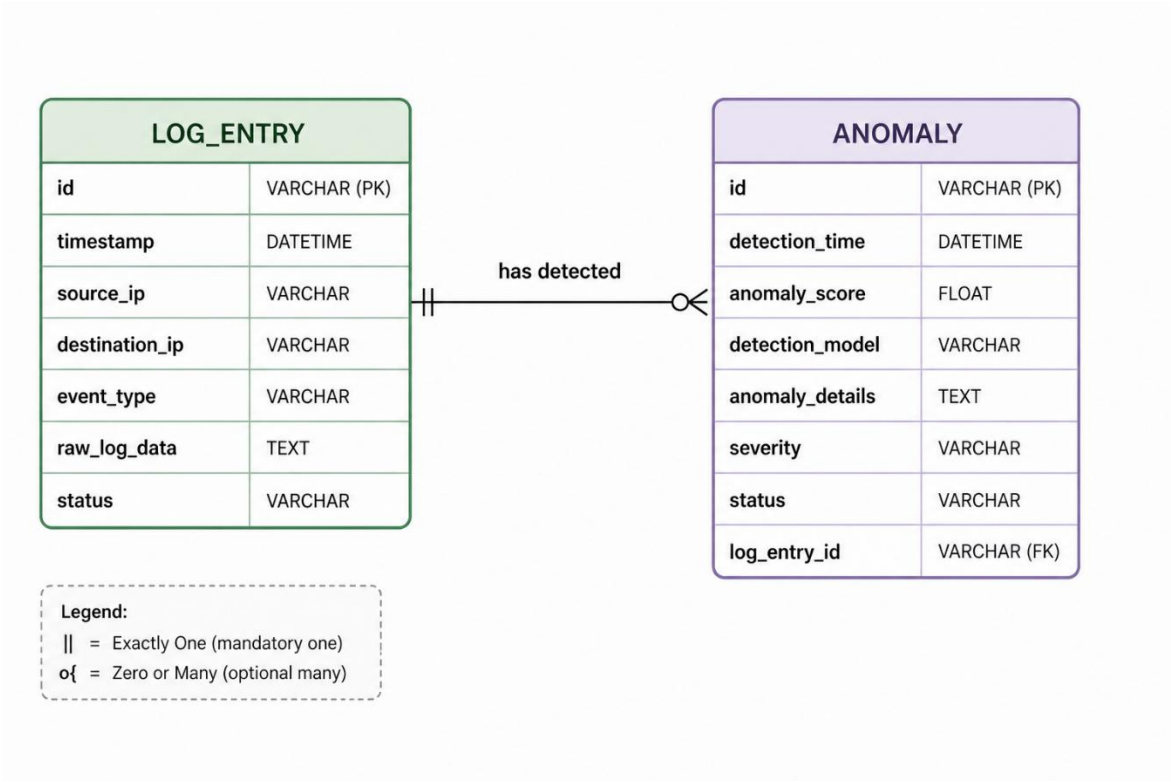
4.4. Architectural Diagram



5. Database Design & Data Modeling

5.1. ER Diagram (Entity-Relationship Diagram)

The Mini SIEM system will primarily deal with log data and detected anomalies. A simplified ERD is presented below, focusing on the core entities and their relationships.



5.2. Logical & Physical Schema

Given the nature of security logs and anomaly data, a time-series database is highly recommended for efficient storage and retrieval, especially for real-time monitoring and analysis. However, for a prototype, a relational database can be used. Below is a logical and physical schema representation.

Logical Schema

- **LogEntry**
  - **id:** Unique identifier for the log entry.
  - **timestamp:** The time when the log entry was generated.
  - **source_ip:** Source IP address of the event.
  - **destination_ip:** Destination IP address of the event.
  - **event_type:** Type of event (e.g., login, network connection, file access).

    - **raw_log_data:** The original, raw log string or JSON.
    - **status:** Processing status of the log entry (e.g., 'processed', 'pending').
- **Anomaly**
    - **id:** Unique identifier for the detected anomaly.
    - **detection_time:** The time when the anomaly was detected.
    - **anomaly_score:** A numerical score indicating the degree of anomaly.
    - **detection_model:** The machine learning model that detected the anomaly (e.g., 'Isolation Forest', 'OC-SVM', 'Autoencoder').
    - **anomaly_details:** Detailed information about the anomaly, potentially including features that contributed to its detection.
    - **severity:** The severity level of the anomaly (e.g., 'low', 'medium', 'high', 'critical').
    - **status:** Current status of the anomaly (e.g., 'new', 'investigating', 'resolved', 'false_positive').
    - **log_entry_id:** Foreign key referencing the LogEntry that caused the anomaly.

Physical Schema (Example using PostgreSQL/MySQL)

Table: **log_entries**

Column Name	Data Type	Constraints	Description
id	VARCHAR(255)	PRIMARY KEY	Unique identifier for the log entry
timestamp	TIMESTAMP	NOT NULL	Time of log generation
source_ip	VARCHAR(45)	NULLABLE	Source IP address
destination_ip	VARCHAR(45)	NULLABLE	Destination IP address
event_type	VARCHAR(100)	NOT NULL	Type of event
raw_log_data	TEXT	NOT NULL	Original raw log data
status	VARCHAR(50)	DEFAULT 'pending'	Processing status

Table: **anomalies**

Column Name	Data Type	Constraints	Description
id	VARCHAR(255)	PRIMARY KEY	Unique identifier for the anomaly
detection_time	TIMESTAMP	NOT NULL	Time when anomaly was detected
anomaly_score	FLOAT	NOT NULL	Numerical score of anomaly
detection_model	VARCHAR(100)	NOT NULL	ML model used for detection
anomaly_details	TEXT	NULLABLE	Detailed anomaly information
severity	VARCHAR(50)	NOT NULL	Severity level of anomaly
status	VARCHAR(50)	DEFAULT 'new'	Current status of anomaly
log_entry_id	VARCHAR(255)	FOREIGN KEY REFERENCES log_entries(id)	Foreign key to the associated log entry

5.3. Normalization Considerations

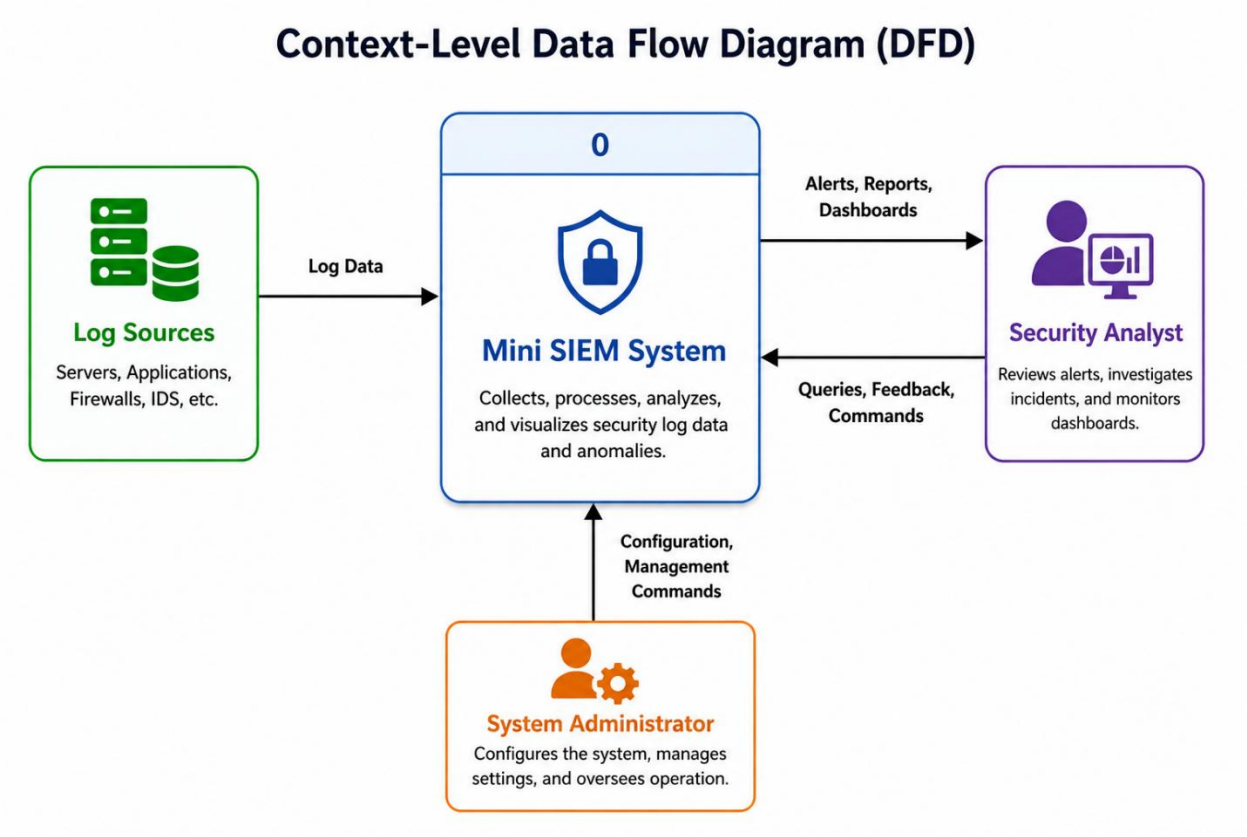
The proposed schema is largely in Third Normal Form (3NF). Each table has a primary key, and all non-key attributes are dependent on the primary key, and there are no transitive dependencies. Further normalization might involve separating event_type into its own lookup table if the number of distinct event types is large and requires frequent updates or additional attributes. However, for the current scope, the chosen structure balances normalization with query performance for log and anomaly data.

6. Data Flow & System Behavior

6.1. DFD (Data Flow Diagram)

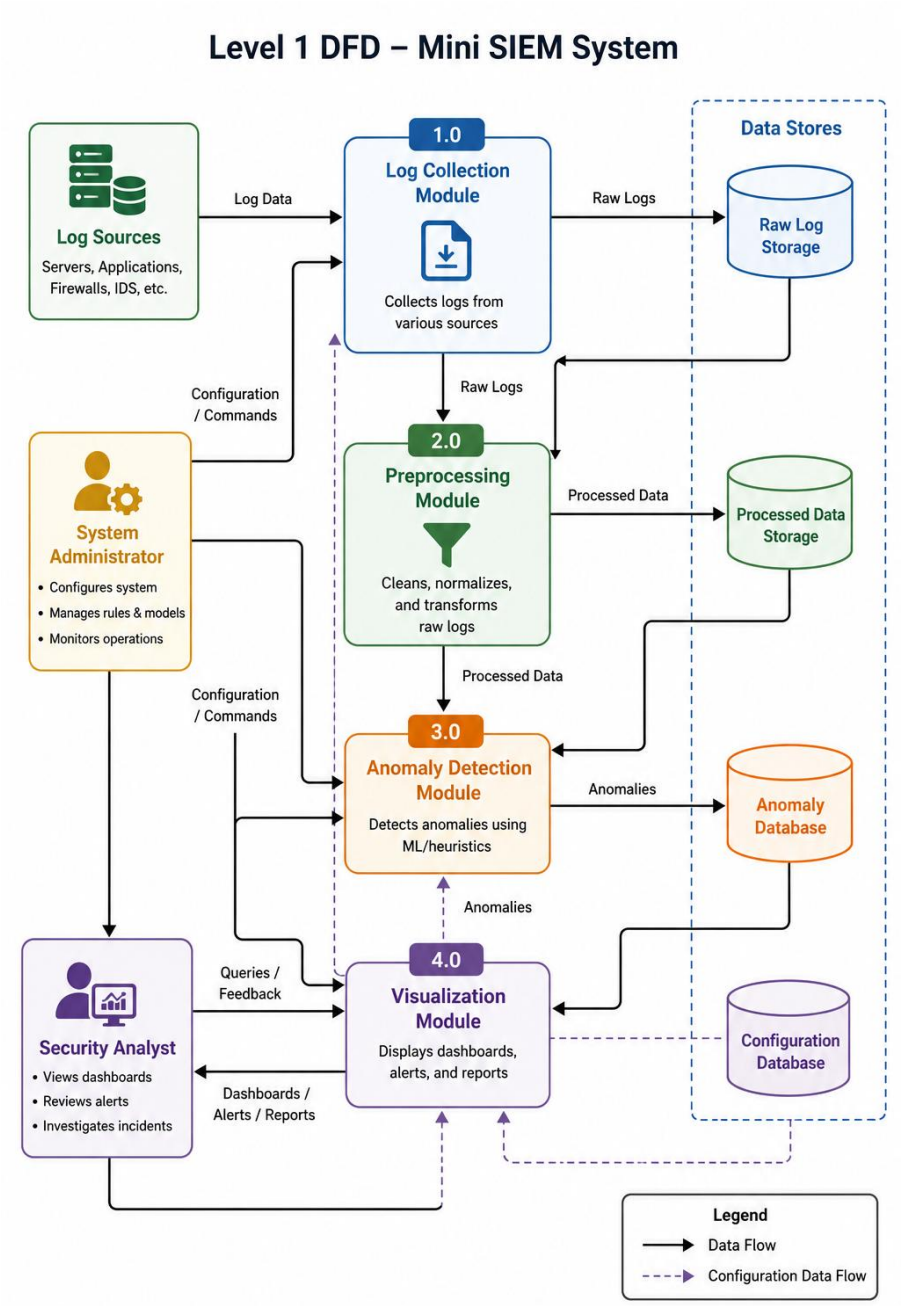
6.1.1. Context-Level DFD

The context-level Data Flow Diagram (DFD) illustrates the Mini SIEM system as a single process, showing its interactions with external entities such as Log Sources, Security Analyst, and System Administrator.



6.1.2. Level 1 DFD

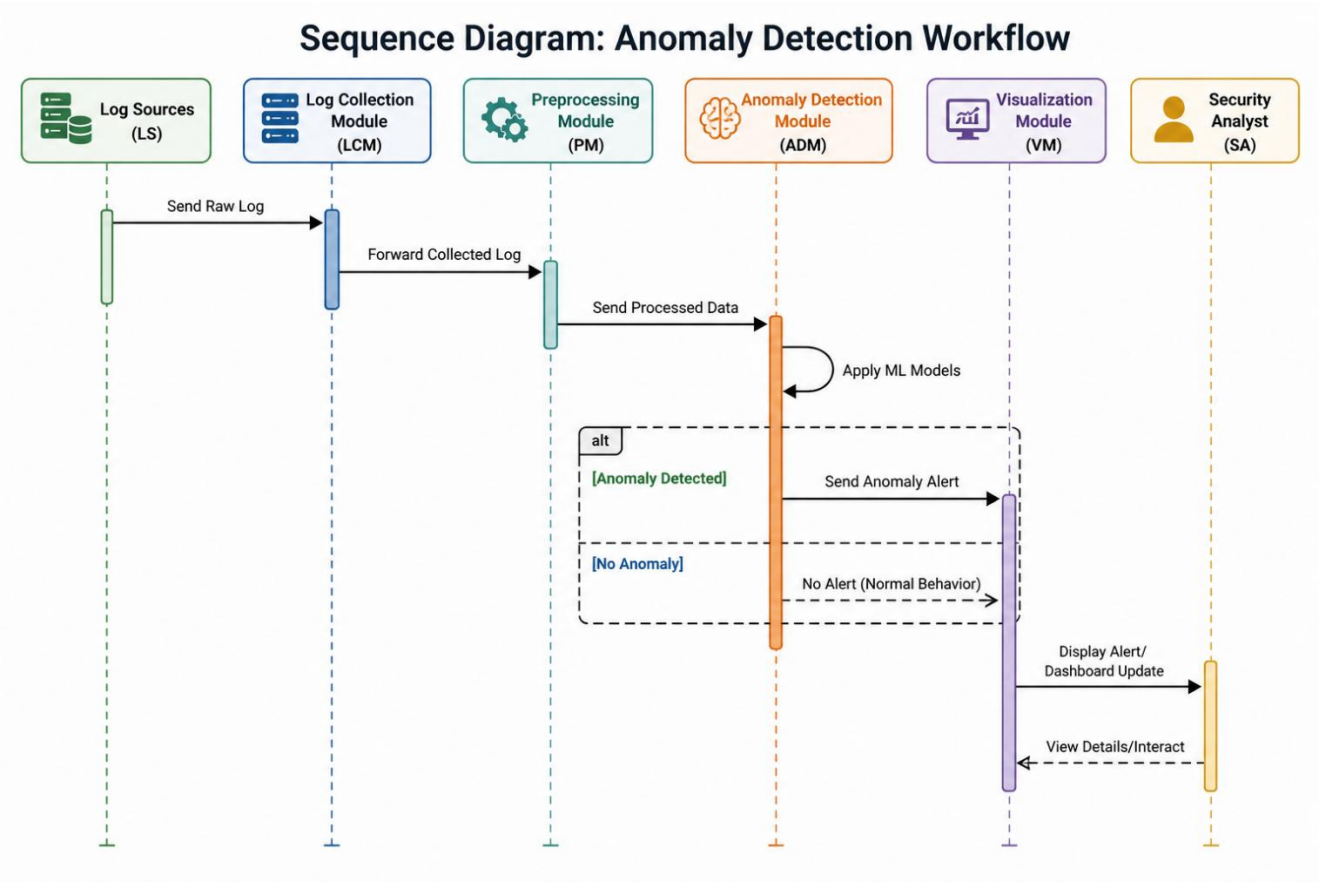
The Level 1 DFD breaks down the Mini SIEM system into its major processes: Log Collection, Preprocessing, Anomaly Detection, and Visualization, and shows the data flows between them and with external entities.



6.2. Sequence Diagrams

6.2.1. Sequence Diagram: Anomaly Detection Workflow

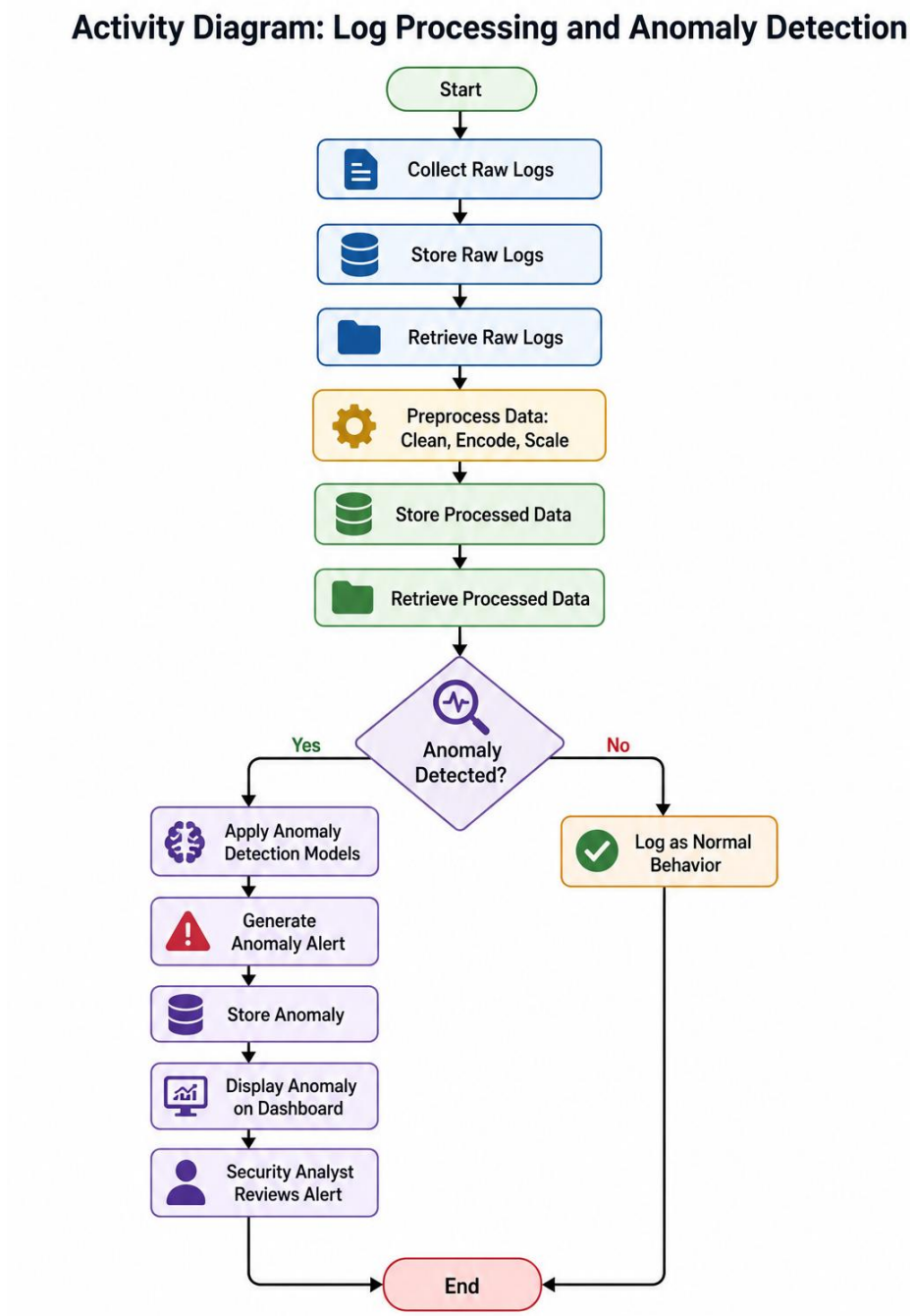
This sequence diagram illustrates the flow of data and interactions during the anomaly detection process, from log ingestion to alert display.



6.3. Activity Diagram

6.3.1. Activity Diagram: Log Processing and Anomaly Detection

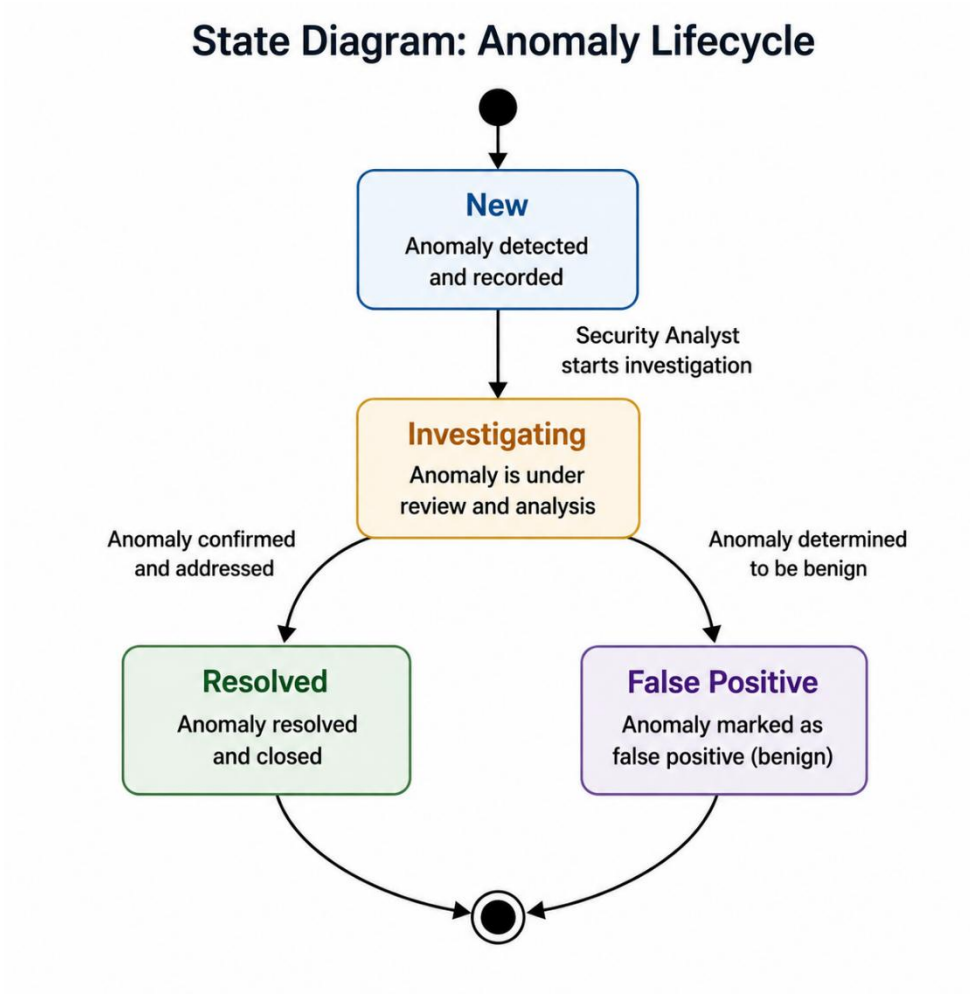
This activity diagram visualizes the workflow of log processing and anomaly detection within the Mini SIEM system.



6.4. State Diagram

6.4.1. State Diagram: Anomaly Lifecycle

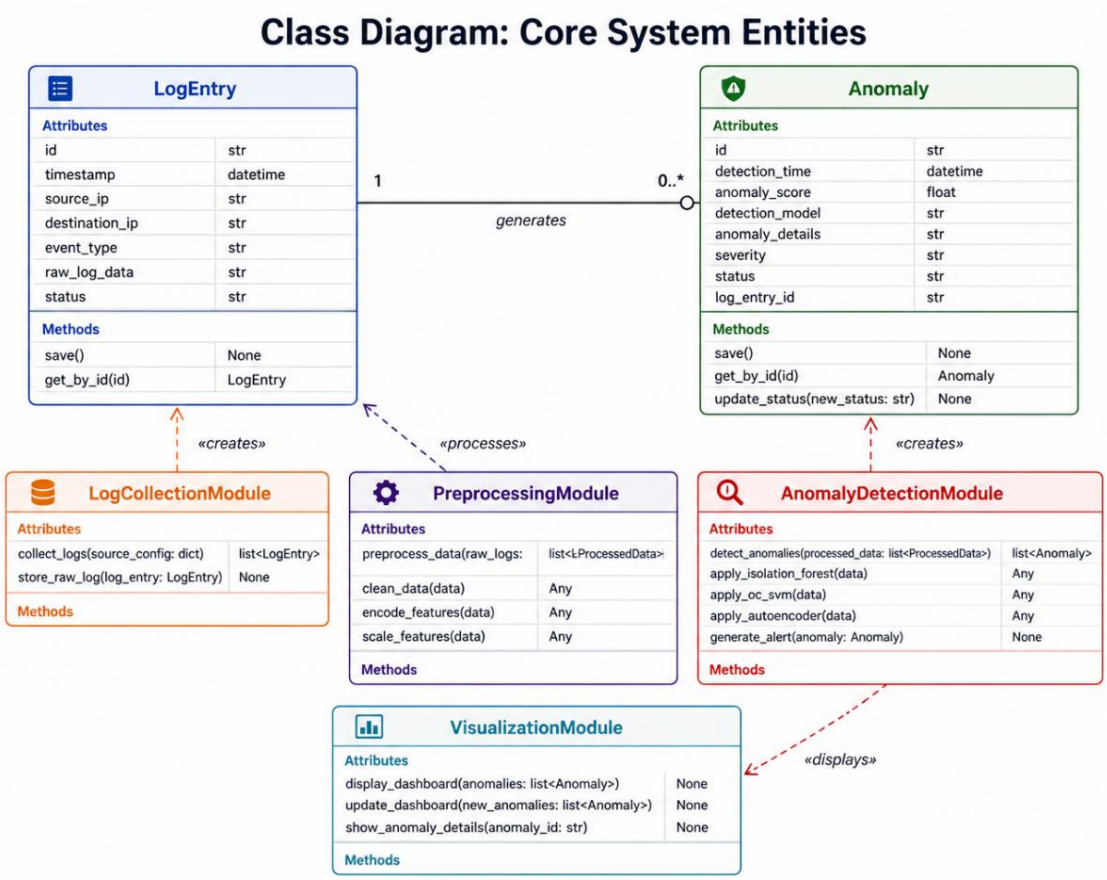
This state diagram represents the different states an anomaly can transition through within the system.



6.5. Class Diagram

6.5.1. Class Diagram: Core System Entities

This class diagram defines the structure of the system by showing core classes, their attributes, methods, and relationships.



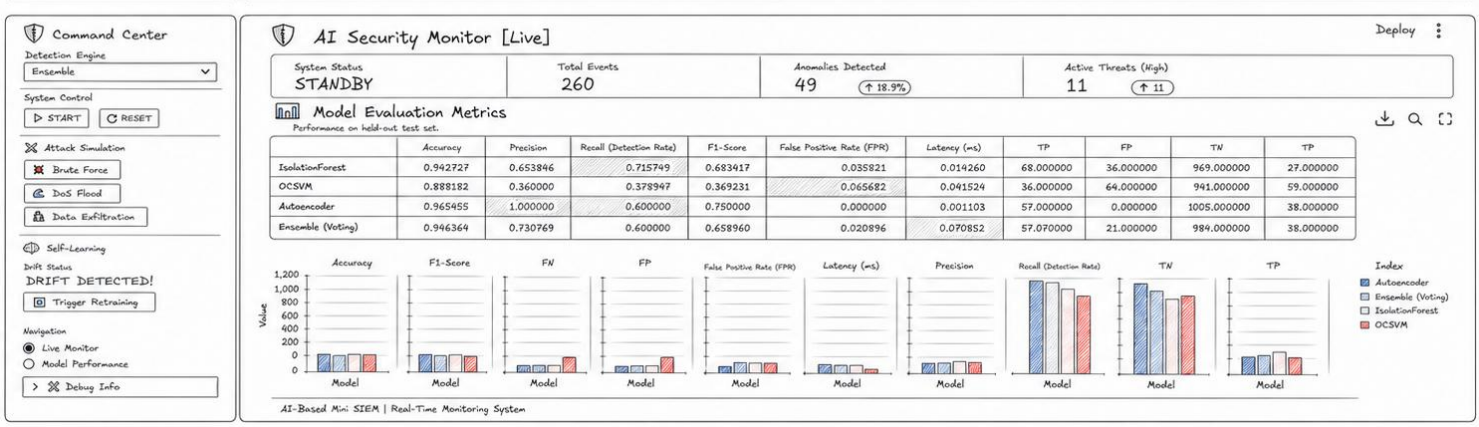
7. UI/UX Design & Prototyping

7.1. Wireframes & Mockups

The user interface for the Mini SIEM system will primarily consist of an interactive dashboard, designed to provide security analysts with a clear and concise overview of detected anomalies and system status. Key screens will include:

- **Dashboard Overview**: A landing page displaying a summary of recent alerts, anomaly trends, and system health indicators. This will include widgets for top anomalous events, a timeline of detections, and a quick-view of log ingestion rates.
- **Anomaly Details View**: A dedicated screen for each anomaly, presenting comprehensive information such as anomaly score, detection model, affected entities (e.g., source IP, destination IP), raw log data, and suggested investigative steps.
- **Configuration Panel**: An interface for System Administrators to manage log sources, configure model parameters, and set alert thresholds.

(Note: Detailed wireframes and mockups would be developed in a subsequent design phase, but the conceptual layout focuses on intuitive navigation and information hierarchy.)



7.2. UI/UX Guidelines

To ensure a consistent, efficient, and user-friendly experience, the following UI/UX guidelines will be adhered to:

- Clarity and Simplicity: The interface will prioritize clear presentation of critical information, avoiding clutter and unnecessary complexity. Data visualizations will be straightforward and easy to interpret.
- Consistency: Design elements, interaction patterns, and terminology will be consistent across all screens and modules to reduce cognitive load and improve learnability.
- Feedback and Responsiveness: The system will provide immediate and clear feedback to user actions, indicating progress, success, or errors. The interface will be responsive, adapting to various screen sizes and devices.
- Accessibility: Design considerations will include accessibility standards to ensure the system is usable by individuals with diverse needs. This includes sufficient color contrast, keyboard navigation support, and clear labeling.
- Data Visualization Best Practices: Anomalies and trends will be visualized using appropriate chart types (e.g., line charts for trends, bar charts for distributions, heatmaps for density) with clear legends and interactive elements for drill-down capabilities.
- Actionable Insights: The dashboard will not only display data but also aim to provide actionable insights, guiding security analysts towards potential threats and facilitating rapid response.
- Error Prevention and Recovery: The system will incorporate mechanisms to prevent common errors and provide clear guidance for recovery when errors do occur.

8. System Deployment & Integration

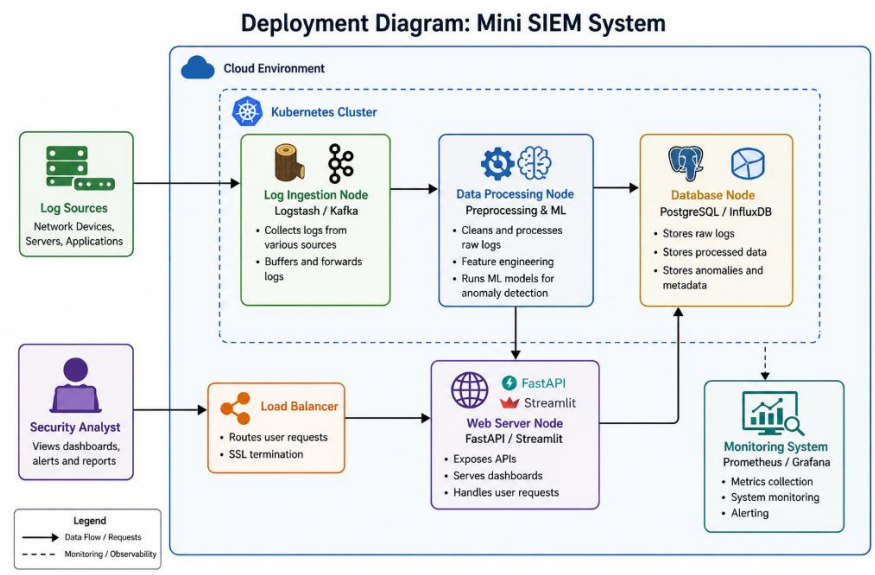
8.1. Technology Stack

The Mini SIEM system will leverage a modern and robust technology stack, primarily centered around Python for its data processing and machine learning capabilities. The proposed stack is designed for flexibility, scalability, and ease of development.

Category	Technology/Tool	Purpose
Programming	Python	Core language for data processing, machine learning, and backend logic.
Machine Learning	Scikit-learn	Implementation of traditional unsupervised ML models (Isolation Forest, OC-SVM).
Deep Learning	PyTorch	Framework for developing and training the Autoencoder Neural Network.
Data Processing	Pandas	Efficient data manipulation and analysis for log preprocessing.
Web Framework (Backend)	FastAPI	(Future consideration) For wrapping Python models into a high-performance REST API.
Frontend/Visualization	Streamlit	Rapid prototyping and development of the interactive dashboard.
Database	PostgreSQL/MySQL	(Prototype) Relational database for storing log metadata and anomaly details.
Time-Series Database	InfluxDB	(Future consideration) Optimized for storing and querying time-stamped data, ideal for logs and metrics.
Log Ingestion	Logstash/Kafka	(Future consideration) Real-time log collection and streaming.
Containerization	Docker	Packaging application components for consistent deployment across environments.
Orchestration	Kubernetes	(Future consideration) Managing and scaling containerized applications.

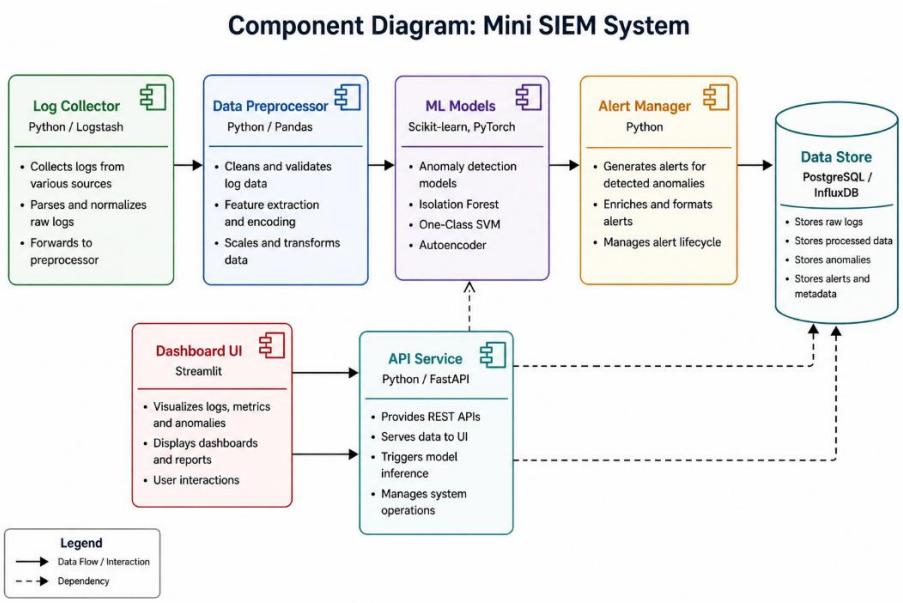
8.2. Deployment Diagram

The deployment diagram illustrates how the software components of the Mini SIEM system are distributed across hardware nodes in a production environment. For the prototype, all components might run on a single server or containerized environment. For real-world deployment, a distributed setup is envisioned.



8.3. Component Diagram

The component diagram shows the high-level software components of the Mini SIEM system and their dependencies. This view focuses on the logical structure and relationships between major functional blocks.



9. Additional Deliverables

9.1. API Documentation

If the Mini SIEM system evolves to include external integrations or a dedicated API service (as suggested in the future considerations for the API Gateway), comprehensive API documentation will be provided. This documentation will detail:

- Endpoints: A list of all available API endpoints, including their HTTP methods (GET, POST, PUT, DELETE).
- Request/Response Formats: Clear descriptions of expected request payloads and response structures, typically in JSON format.
- Authentication: Details on how to authenticate with the API (e.g., API keys, OAuth tokens).
- Error Codes: A catalog of possible error codes and their meanings.
- Usage Examples: Code snippets or curl commands demonstrating how to interact with each endpoint.

Tools like Swagger/OpenAPI Specification will be utilized to generate interactive and machine-readable API documentation, ensuring ease of use for developers integrating with the Mini SIEM system.

9.2. Testing & Validation

A robust testing and validation strategy will be implemented to ensure the reliability, accuracy, and performance of the Mini SIEM system. This will include:

- Unit Tests: Individual components and functions (e.g., data preprocessing functions, anomaly detection model logic) will be tested in isolation to ensure they perform as expected.
- Integration Tests: Tests will verify the interactions and data flow between different modules (e.g., Log Collection Module with Preprocessing Module, Anomaly Detection Module with Visualization Module).
- User Acceptance Testing (UAT): Security analysts and system administrators will perform UAT to validate that the system meets their operational requirements and user experience expectations in a simulated production environment.
- Performance Testing: The system will be tested under various load conditions to assess its scalability and responsiveness, particularly concerning log ingestion rates and anomaly detection latency.
- Model Validation: The anomaly detection models will be rigorously evaluated using appropriate metrics (e.g., precision, recall, F1-score, AUC-ROC) on diverse datasets, including both normal and anomalous data, to ensure their effectiveness and minimize false positives.

9.3. Deployment Strategy

The deployment strategy for the Mini SIEM system will focus on automation, scalability, and high availability. While the initial prototype might be deployed on a single instance, a production-grade deployment will follow these principles:

- Containerization: All system components will be containerized using Docker, ensuring consistent environments across development, testing, and production.
- Orchestration: Kubernetes will be used for container orchestration, enabling automated deployment, scaling, and management of the microservices.
- CI/CD Pipeline: A Continuous Integration/Continuous Deployment (CI/CD) pipeline will automate the build, test, and deployment processes, ensuring rapid and reliable delivery of updates and new features.
- Infrastructure as Code (IaC): Tools like Terraform or Ansible will be used to define and manage the infrastructure (e.g., cloud resources, Kubernetes clusters) as code, promoting consistency and reproducibility.
- Monitoring and Logging: Comprehensive monitoring (e.g., Prometheus, Grafana) and centralized logging (e.g., ELK stack) will be implemented to track system health, performance, and detect operational issues in real-time.
- Rollback Capability: The deployment process will include mechanisms for quick and safe rollbacks to previous stable versions in case of deployment failures or critical issues.
- Scalability Considerations: Each microservice will be designed to scale independently based on demand, allowing for efficient resource utilization and performance optimization.

References

[1] Project Idea Document: AI-Based Security Monitoring and Anomaly Detection System (Mini SIEM). (2026). *Provided by User.*